Installing OpenClaw on Your Turing Pi RK1 Kubernetes Cluster

with a Discord Front End

ARM64 • K3s • Longhorn • MetalLB • Claude AI

October 2026 — Version 1.1

Table of Contents

- 1. Overview
  - 1.1 Prerequisites
  - 1.2 What You Will Need
  - 1.3 Architecture
- 2. Verify ARM64 Image Compatibility
- 3. Set Up the Anthropic API Key
  - 3.1 Create an API Key
  - 3.2 Pricing Notes
- 4. Create and Configure the Discord Bot
  - 4.1 Create a Discord Application
  - 4.2 Enable Privileged Gateway Intents
  - 4.3 Generate the Bot Invite URL
- 5. Create Kubernetes Namespace and Secrets
  - 5.1 Create the Namespace
  - 5.2 Create the Secrets
- 6. Deploy OpenClaw
  - 6.1 Create the Deployment Manifest
  - 6.2 Apply the Manifest
  - 6.3 Watch the Deployment
- 7. Verify Discord Connection
  - 7.1 Check the Logs
  - 7.2 Troubleshooting: Discord Error 4014
- 8. Switch the Model to Claude Sonnet (Recommended)
  - 8.1 Update the Configuration
  - 8.2 Restart to Apply
  - 8.3 Critical: Config Overwrite Behavior
- 9. Security Hardening

- 10. Enable Web Search (Brave API)
  - 10.1 Get a Brave Search API Key
  - 10.2 Add the Key to Kubernetes Secrets
- 11. Connect Signal Messaging (Optional)
  - 11.1 Prerequisites
  - 11.2 Register Signal with a Google Voice Number
  - 11.3 Enable Signal in OpenClaw
  - 11.4 Link the Device
- 12. Operational Notes
  - 12.1 Persistent State
  - 12.2 Updating OpenClaw
  - 12.3 Viewing Logs
  - 12.4 Web UI Access
  - 12.5 Configuration File
  - 12.6 Resource Usage
- 13. Cleanup
- 14. Quick Reference
- 15. Troubleshooting Summary

---

1. Overview

OpenClaw is an open-source AI personal assistant that connects to messaging platforms like Discord, Telegram, and WhatsApp. It uses large language models (Anthropic Claude, OpenAI, etc.) to respond to messages, execute tasks, browse the web, manage files, and run code autonomously.

This guide walks through deploying OpenClaw on a Turing Pi 2.5 K3s cluster with RK1 ARM64 compute modules, using Discord as the messaging front end and Anthropic Claude as the LLM provider. The deployment uses raw Kubernetes manifests rather than Helm charts for transparency and control.

1.1 Prerequisites

This guide assumes the following are already configured on your Turing Pi 2.5:

- **K3s HA cluster** — 3 server nodes + 1 worker node running K3s with embedded etcd
- **Longhorn** — distributed storage provisioner installed and healthy, **with the multipathd blacklist in place** (vendor `IET`, product `VIRTUAL-DISK`; see the build guide §3.7). Without it, multipathd grabs the Longhorn iSCSI device and the PVC fails with EIO. On this cluster that caused a 35-day OpenClaw crashloop.
- **MetalLB** — configured with an IPAddressPool and L2Advertisement for LoadBalancer services
- **kubectl access** — from any node or workstation with cluster credentials

1.2 What You Will Need

- **Anthropic API key** — from console.anthropic.com (pay-as-you-go, separate from Claude chat subscription)
- **Discord bot token** — from discord.com/developers/applications
- **Discord server** — where you will invite the bot

1.3 Architecture

The deployment consists of a single-replica Deployment running the official OpenClaw container image, a Longhorn PersistentVolumeClaim for state persistence, and a MetalLB LoadBalancer Service for optional web UI access. The Discord bot connects outbound from the pod to Discord's gateway, so no inbound network access is required for messaging.

Component	**Details**
Container Image	ghcr.io/openclaw/openclaw:latest (multi-arch: amd64 + arm64)
Namespace	openclaw
Storage	10 Gi Longhorn PVC at /home/node/.openclaw
Service	LoadBalancer (MetalLB) on port 18789
LLM Provider	Anthropic Claude (Sonnet; live: `anthropic/claude-sonnet-4-6`)
Messaging	Discord bot (outbound WebSocket connection)
Container User	UID 1000 (node)
Resource Limits	250m–1000m CPU, 512 Mi–2 Gi RAM

2. Verify ARM64 Image Compatibility

Before deploying, confirm that the official OpenClaw container image includes an ARM64 variant. The Turing Pi RK1 modules use Rockchip RK3588 ARM64 processors, and running an amd64-only image will fail with an exec format error.

```
# Check the multi-arch manifest
docker manifest inspect ghcr.io/openclaw/openclaw:latest
```

Look for a platform entry with architecture: arm64. The output should include both amd64 and arm64 manifests.

> ✅ **CHECKPOINT: ARM64 image available Run:** docker manifest inspect ghcr.io/openclaw/openclaw:latest | grep arm64 **Expected:** *Output should contain: "architecture": "arm64"*

3. Set Up the Anthropic API Key

OpenClaw uses the Anthropic API to communicate with Claude. This is a separate service from the Claude chat subscription at claude.ai and uses pay-as-you-go pricing.

3.1 Create an API Key

1. Go to **console.anthropic.com** and sign in (or create an account)
2. Navigate to **API Keys** in the left sidebar
3. Click **Create Key** and give it a descriptive name (e.g., openclaw-k3s)
4. Copy the key immediately — it will not be shown again

3.2 Pricing Notes

The Anthropic API charges per token consumed. There is no monthly subscription fee: you only pay for what you use. Prices differ by model (Opus > Sonnet > Haiku) and change over time, so check the current rates on Anthropic's pricing page before choosing a model: https://www.anthropic.com/pricing (API section) or https://docs.anthropic.com/en/docs/about-claude/pricing.

> *Note (October 2026):* this section used to have a price table. It was removed because the figures were out of date. Always check the pricing page instead.

Important: OpenClaw defaults to Claude Opus (the most expensive model). Section 8 covers how to switch to Sonnet after deployment.

4. Create and Configure the Discord Bot

4.1 Create a Discord Application

5. Go to **discord.com/developers/applications**
6. Click **New Application** and name it (e.g., “Cheshire” or “OpenClaw”)
7. Go to the **Bot** section in the left sidebar
8. Click **Reset Token** and copy the bot token

4.2 Enable Privileged Gateway Intents

This is critical — without the Message Content Intent, the bot will crash with Discord error 4014.

9. In the Bot section, scroll to **Privileged Gateway Intents**
10. Enable **Message Content Intent** (toggle ON)
11. Optionally enable **Presence Intent** and **Server Members Intent**
12. Click **Save Changes**

> ✅ **CHECKPOINT: Message Content Intent enabled Run:** Visually confirm all three Privileged Gateway Intents are toggled ON with green indicators in the Discord Developer Portal **Expected:** *All toggles green, Save Changes clicked*

4.3 Generate the Bot Invite URL

13. Go to **OAuth2 → URL Generator** in the left sidebar
14. Under Scopes, check **bot**
15. Under Bot Permissions, check **Send Messages**, **Read Message History**, and **View Channels**
16. Copy the generated URL and open it in your browser
17. Select your Discord server and click **Authorize**

> ✅ **CHECKPOINT: Bot appears in Discord server Run:** Check the member list in your Discord server **Expected:** *Bot appears as offline member (it will come online after deployment)*

5. Create Kubernetes Namespace and Secrets

5.1 Create the Namespace

```
kubectl create namespace openclaw
```

> ✅ **CHECKPOINT: Namespace exists Run:** kubectl get namespace openclaw **Expected:** *Status: Active*

5.2 Create the Secrets

Store your API key and Discord token as a Kubernetes Secret. Replace the placeholder values with your actual keys:

```
kubectl create secret generic openclaw-secrets -n openclaw \
  --from-literal=ANTHROPIC_API_KEY='sk-ant-your-key-here' \
  --from-literal=DISCORD_BOT_TOKEN='your-discord-bot-token-here'
```

> ✅ **CHECKPOINT: Secret created with both keys Run:** kubectl get secret openclaw-secrets -n openclaw -o jsonpath='{.data}' | tr ',' '\n' **Expected:** *Output shows both ANTHROPIC_API_KEY and DISCORD_BOT_TOKEN as base64-encoded values*

6. Deploy OpenClaw

6.1 Create the Deployment Manifest

Create a file called openclaw-deploy.yaml with the following content. This includes a PersistentVolumeClaim (Longhorn), a Deployment with an init container for permissions, and a

LoadBalancer Service (MetalLB).

Key design decisions explained:

- **Init container:** Longhorn PVCs mount as root-owned. OpenClaw runs as UID 1000 (node user). The Alpine init container runs chown -R 1000:1000 to fix ownership before the main container starts.
- **No liveness/readiness probes:** OpenClaw's gateway binds to 127.0.0.1 only and does not accept external HTTP requests. Standard Kubernetes probes cannot reach it. The Discord connection is outbound, so probe failures are harmless for the bot functionality.
- **Recreate strategy:** OpenClaw is stateful and single-instance. Recreate ensures the old pod releases the Longhorn volume before the new pod starts.
- **Security context:** The pod runs as UID 1000 with seccomp RuntimeDefault profile, dropped Linux capabilities, and allowPrivilegeEscalation: false. This prevents container escape and restricts syscalls to a safe default set.

```
cat <<'EOF' > openclaw-deploy.yaml
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: openclaw-data
  namespace: openclaw
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: longhorn
  resources:
    requests:
      storage: 10Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: openclaw
  namespace: openclaw
  labels:
    app: openclaw
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: openclaw
  template:
    metadata:
      labels:
        app: openclaw
    spec:
      initContainers:
        - name: fix-permissions
          image: alpine:latest
          command: ["sh", "-c", "chown -R 1000:1000 /data"]
          volumeMounts:
            - name: data
              mountPath: /data
      securityContext:
```

```
        runAsUser: 1000
        runAsGroup: 1000
        fsGroup: 1000
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: openclaw
          image: ghcr.io/openclaw/openclaw:latest
          securityContext:
            allowPrivilegeEscalation: false
            runAsNonRoot: true
            capabilities:
              drop:
                - ALL
          ports:
            - containerPort: 18789
              name: gateway
          envFrom:
            - secretRef:
                name: openclaw-secrets
          env:
            - name: NODE_ENV
              value: "production"
          volumeMounts:
            - name: data
              mountPath: /home/node/.openclaw
          resources:
            requests:
              cpu: 250m
              memory: 512Mi
            limits:
              cpu: 1000m
              memory: 2Gi
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: openclaw-data
---
apiVersion: v1
kind: Service
metadata:
  name: openclaw
```

```
  namespace: openclaw
  annotations:
    metallb.io/address-pool: default-pool
    metallb.io/loadBalancerIPs: 192.168.4.203
spec:
  type: LoadBalancer
  selector:
    app: openclaw
  ports:
    - name: gateway
      port: 18789
      targetPort: 18789
EOF
```

Note: Adjust the MetalLB IP address (192.168.4.203) and the address-pool name (default-pool) to match your cluster's MetalLB configuration. Check your pool name with: kubectl get ipaddresspool -A

MetalLB pitfalls (from this cluster): - Use the `metallb.io/*` annotations. The older `metallb.universe.tf/*` keys are deprecated (migrated here 2026-04-03). - This Service is SingleStack IPv4 by default. If you request an IPv4 *and* an IPv6 address on a SingleStack Service (`192.168.4.203,fd00::203`), MetalLB cannot satisfy it and EXTERNAL-IP stays `<pending>`. That happened here for 37 days. For both addresses, also set `spec.ipFamilyPolicy: PreferDualStack` and `ipFamilies: [IPv4, IPv6]`.

6.2 Apply the Manifest

```
kubectl apply -f openclaw-deploy.yaml
```

6.3 Watch the Deployment

```
kubectl get pods -n openclaw -w
```

You should see the init container (fix-permissions) run first, then the main container (openclaw) start. The image is approximately 1.1 GB, so the first pull may take 30–60 seconds.

> ✅ **CHECKPOINT: Pod is Running Run:** kubectl get pods -n openclaw **Expected:** *STATUS: Running, READY: 1/1 (may show 0/1 briefly during startup)*
>
> ✅ **CHECKPOINT: No permission errors in logs Run:** kubectl logs -n openclaw deployment/openclaw -c openclaw --tail=20 | grep -i 'EACCES|permission' **Expected:** *No output (no permission errors). If you see EACCES errors, the init container did not run properly.*

7. Verify Discord Connection

7.1 Check the Logs

```
kubectl logs -n openclaw deployment/openclaw -c openclaw --tail=15
```

Look for these key lines in the output:

- [discord] logged in to discord as \<bot-id> — confirms successful Discord connection
- [gateway] agent model: anthropic/claude-opus-4-6 — confirms the LLM provider is configured
- [gateway] listening on ws://127.0.0.1:18789 — confirms the gateway started (localhost binding is normal)

7.2 Troubleshooting: Discord Error 4014

If you see the following error, the Message Content Intent is not enabled:

```
[discord] gateway: WebSocket connection closed with code 4014
[discord] gateway error: Error: Fatal Gateway error: 4014
```

Fix: Return to the Discord Developer Portal → Bot → Privileged Gateway Intents and confirm Message Content Intent is toggled ON. Click Save Changes, then restart the pod:

```
kubectl rollout restart deployment/openclaw -n openclaw
```

> ✅ **CHECKPOINT: Discord bot responds Run:** Send a message in your Discord server: @YourBotName hello **Expected:** *Bot responds with an introduction message within 10–20 seconds. If no response, check logs for errors.*

8. Switch the Model to Claude Sonnet (Recommended)

By default, OpenClaw uses Claude Opus, which costs more per token than Sonnet (see the pricing page linked in Section 3.2). For personal/home lab use, Sonnet provides excellent quality at lower cost. The live deployment on this cluster runs `anthropic/claude-sonnet-4-6` (as of 2026-09-26).

8.1 Update the Configuration

This command sets the model and the Discord group policy. It also makes sure `gateway.mode` is set, which current OpenClaw releases require (see Section 8.3):

```
kubectl exec -n openclaw deployment/openclaw -c openclaw -- \
  node -e "
    const fs = require('fs');
    const cfg = JSON.parse(fs.readFileSync('/home/node/.openclaw/openclaw.json'));
    cfg.agents = cfg.agents || {};
    cfg.agents.defaults = cfg.agents.defaults || {};
    cfg.agents.defaults.model = { primary: 'anthropic/claude-sonnet-4-5-20250929' };
    cfg.channels = cfg.channels || {};
    cfg.channels.discord = cfg.channels.discord || {};
    cfg.channels.discord.groupPolicy = 'open';
    cfg.gateway = cfg.gateway || {};
    cfg.gateway.mode = cfg.gateway.mode || 'local';
    if (cfg.meta) delete cfg.meta.configMode;  // removed from the schema; a stale key
breaks startup
    fs.writeFileSync('/home/node/.openclaw/openclaw.json', JSON.stringify(cfg, null,
2));
    console.log('Config updated: Sonnet model, open groupPolicy, gateway.mode');
  "
```

8.2 Restart to Apply

```
kubectl rollout restart deployment/openclaw -n openclaw
```

Wait 30–40 seconds for the pod to stabilize, then verify:

```
kubectl logs -n openclaw deployment/openclaw -c openclaw --tail=10 | grep model
```

> ✅ **CHECKPOINT: Model is Sonnet Run:** kubectl logs -n openclaw deployment/openclaw -c openclaw --tail=10 | grep model **Expected:** *Output shows: [gateway] agent model: anthropic/claude-sonnet-4-5-20250929*
>
> ✅ **CHECKPOINT: Pod is stable (no crash loops) Run:** kubectl get pods -n openclaw **Expected:** *STATUS: Running, RESTARTS: 0 (or 1 from the rollout). If RESTARTS keeps increasing, check logs for Discord 4014 error.*

8.3 Critical: Config Overwrite Behavior

Warning: OpenClaw can overwrite your configuration file on restart. When the pod detects a change in environment variables (such as adding a new API key to the Kubernetes Secret), the startup process may regenerate openclaw.json from scratch, discarding any manual edits you have made.

When this happens, you will see a log line like:

```
Config overwrite: /home/node/.openclaw/openclaw.json (sha256 ... -> ...)
```

The overwrite resets settings to defaults, which means your model choice (Sonnet) reverts to Opus, the Discord groupPolicy changes from open to allowlist (breaking Discord bot responses), and any installed skills are removed.

`meta.configMode` no longer exists. Earlier versions of this guide set `meta.configMode = 'merge'` to stop the overwrite. Newer OpenClaw releases dropped that key from the config schema, and a leftover `configMode` stopped the pod from starting (removed on this cluster 2026-04-02). Don't set it. If your config still has it, delete it; the Section 8.1 command does that.

If your bot stops responding after a restart, check for this overwrite message in the logs. To recover, re-run the command in Section 8.1 and restart the deployment. Keep a copy of a known-good `openclaw.json` (for example `openclaw.json.bak-<date>` on the PVC) before any change to the Secret or the image.

After an image upgrade (especially with `:latest`), the config on the PVC can be older than what the new release expects. On this cluster the 2026.2.x → 2026.9.x jump (2026-09-26) caused a crashloop for two reasons: 1. `existing config is missing gateway.mode` → add `"gateway": {"mode": "local"}` (the Section 8.1 command does this). 2. `Auth profile store … requires legacy credential migration` → scale the Deployment to 0, run `openclaw doctor --fix --non-interactive` in a one-off pod using the same image, Secret and PVC (command `sleep`, then exec the doctor), and scale back to 1. Back up `~/.openclaw` first.

Backup: OpenClaw saves the previous config as openclaw.json.bak before overwriting. You can inspect it with:

```
kubectl exec -n openclaw deployment/openclaw -c openclaw -- \
  cat /home/node/.openclaw/openclaw.json.bak
```

> ✅ **CHECKPOINT: Config matches the current schema Run:** kubectl exec -n openclaw deployment/openclaw -c openclaw -- grep -E '"configMode"|"mode"' /home/node/.openclaw/openclaw.json **Expected:** *No `configMode` line; a `"mode": "local"` line is present (under `gateway`)*

9. Security Hardening

OpenClaw is an AI agent with broad capabilities including shell execution, file system access, and browser automation. Securing the container and its network exposure is essential, especially on a home lab cluster that may share the network with other devices.

10.1 Security Audit Findings

OpenClaw includes a built-in security audit feature. Running the audit on a default deployment surfaces the following areas for attention. Each is addressed in this section.

Finding	Risk	Status	Mitigation
Seccomp Profile	Container can make unrestricted syscalls	**Fixed in manifest**	seccompProfile: RuntimeDefault applied at pod level
No New Privileges	Processes could escalate privileges	**Fixed in manifest**	allowPrivilegeEscalation: false, capabilities dropped
Network Exposure	Gateway binds to localhost only	**Acceptable**	Discord is outbound; web UI via port-forward only
Backup Status	No external backups configured	**Recommended**	Configure Longhorn recurring snapshots (Section 9.4)
Update Policy	No automated update cadence	**Recommended**	Pin image tags or use scheduled rollout restarts (Section 9.5)
CLI Access	openclaw CLI returns permission denied	**Non-issue**	PATH issue inside container; does not affect runtime security

9.2 Pod Security Context (Applied in Manifest)

The deployment manifest in Section 6 includes a hardened security context. These settings are applied automatically when you deploy using the provided YAML:

- **runAsUser: 1000 / runAsGroup: 1000** — container runs as the unprivileged node user, never root
- **fsGroup: 1000** — ensures the Longhorn volume is group-accessible to the container user
- **seccompProfile: RuntimeDefault** — applies the container runtime's default seccomp filter, which blocks dangerous syscalls like reboot, mount, and kernel module operations while allowing normal application behavior
- **allowPrivilegeEscalation: false** — prevents any process inside the container from gaining more privileges than its parent (blocks setuid/setgid binaries)
- **runAsNonRoot: true** — Kubernetes will refuse to start the container if the image tries to run as UID 0
- **capabilities.drop: ALL** — removes all Linux capabilities (NET_RAW, SYS_ADMIN, etc.), limiting the container to pure unprivileged operation

> ✅ **CHECKPOINT: Security context is applied Run:** kubectl get pod -n openclaw -l app=openclaw -o jsonpath='{.items[0].spec.securityContext}' kubectl get pod -n openclaw -l app=openclaw -o jsonpath='{.items[0].spec.containers[0].securityContext}' **Expected:** *First command shows runAsUser:1000, seccompProfile. Second shows allowPrivilegeEscalation:false, runAsNonRoot:true, drop:[ALL]*

12.3 Network Isolation

OpenClaw's gateway binds to 127.0.0.1 only, which means the MetalLB LoadBalancer IP does not actually expose the web UI. The Discord bot connects outbound to Discord's servers. No inbound connections are required for the bot to function.

For additional network isolation, you can optionally apply a NetworkPolicy that restricts the pod's egress to only DNS and HTTPS:

```
cat <<'EOF' > openclaw-netpol.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: openclaw-egress
  namespace: openclaw
spec:
  podSelector:
    matchLabels:
      app: openclaw
  policyTypes:
    - Egress
  egress:
    # Allow DNS
    - to: []
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
    # Allow HTTPS (Discord API, Anthropic API)
    - to: []
      ports:
        - protocol: TCP
          port: 443
EOF
kubectl apply -f openclaw-netpol.yaml
```

Note: This NetworkPolicy blocks all non-DNS/HTTPS egress. If OpenClaw needs to access other services (e.g., HTTP on port 80, or other cluster services), add additional egress rules.

9.4 Longhorn Backup and Snapshots

Longhorn replicates data across nodes, protecting against single-node failure. However, replication does not protect against accidental deletion, data corruption, or cluster-wide failures. Configure recurring

snapshots for the OpenClaw volume:

```
cat <<'EOF' > openclaw-snapshot-job.yaml
apiVersion: longhorn.io/v1beta2
kind: RecurringJob
metadata:
  name: openclaw-daily-snapshot
  namespace: longhorn-system
spec:
  name: openclaw-daily-snapshot
  task: snapshot
  cron: "0 2 * * *"
  retain: 7
  concurrency: 1
  labels:
    app: openclaw
EOF
kubectl apply -f openclaw-snapshot-job.yaml
```

Then label the OpenClaw PVC to attach the recurring job:

```
kubectl label pvc openclaw-data -n openclaw \
  recurring-job.longhorn.io/source=enabled \
  recurring-job-group.longhorn.io/default=enabled
```

This creates daily snapshots at 2:00 AM and retains the last 7 days. For offsite backups, configure a Longhorn backup target (S3-compatible storage) in the Longhorn UI.

9.5 Update Policy

The deployment uses the :latest image tag, so any rollout restart or pod reschedule pulls the newest version. **Pinning is recommended.** On this cluster, `:latest` silently jumped from 2026.2.25 to 2026.9.x on a reschedule and broke the stored config (see Section 8.3). For a more controlled update policy, consider:

- **Pin to a specific version tag** (e.g., ghcr.io/openclaw/openclaw:2026.9.6) and update deliberately, running `openclaw doctor` after each jump
- **Scheduled rollout restart** — create a CronJob that restarts the deployment weekly to pick up new images:

```
cat <<'EOF' > openclaw-auto-update.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: openclaw-updater
  namespace: openclaw
spec:
  schedule: "0 4 * * 0"  # Every Sunday at 4 AM
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: openclaw-updater
          containers:
            - name: updater
              image: bitnami/kubectl:latest
              command:
                - kubectl
                - rollout
                - restart
                - deployment/openclaw
                - -n
                - openclaw
          restartPolicy: OnFailure
EOF
# Note: requires a ServiceAccount with deployment/patch permissions
```

10. Enable Web Search (Brave API)

By default, OpenClaw cannot search the web. It has a built-in Chromium browser for visiting specific URLs, but web search requires a search API key. OpenClaw uses the Brave Search API, which offers a free tier of 2,000 queries per month.

10.1 Get a Brave Search API Key

18. Go to **brave.com/search/api/** and create an account
19. Subscribe to the **Free** plan (2,000 queries/month, no credit card required)
20. Copy your API key from the dashboard

10.2 Add the Key to Kubernetes Secrets

Use kubectl patch to add the Brave API key to the existing secret without disrupting the other keys:

```
kubectl patch secret openclaw-secrets -n openclaw \
  -p '{"stringData":{"BRAVE_API_KEY":"your-brave-api-key-here"}}'
```

Important: Adding a new environment variable can trigger a config overwrite on restart (see Section 8.3). Back up `openclaw.json` first. If the logs show a `Config overwrite` line after the restart, re-run the Section 8.1 command to restore the model and Discord settings (the old `configMode: merge` setting no longer exists).

12.3 Restart and Verify

```
kubectl rollout restart deployment/openclaw -n openclaw
sleep 30 && kubectl logs -n openclaw deployment/openclaw -c openclaw --tail=5
```

After restart, verify that your settings survived the restart:

> ✅ **CHECKPOINT: Config preserved after restart Run:** kubectl logs -n openclaw deploy/openclaw -c openclaw --tail=15 | grep -E 'model|groupPolicy|overwrite' **Expected:** *Model should still show claude-sonnet-4-5-20250929. No config overwrite line should appear. If you see an overwrite, re-run the Section 8.1 command.*

Test web search by asking the bot in Discord:

```
@Cheshire search the web for K3s longhorn backup best practices
```

> ✅ **CHECKPOINT: Web search works Run:** Ask the bot to search for something in Discord or the web UI **Expected:** *Bot returns results with citations from web sources, not just its training knowledge*

10.4 Optional: Add the Fetch Skill

In addition to web search, you can add the Anthropic fetch MCP server skill, which gives OpenClaw the ability to retrieve and read any URL on demand:

```
kubectl exec -n openclaw deployment/openclaw -c openclaw -- \
  node -e "
    const fs = require('fs');
    const cfg = JSON.parse(fs.readFileSync('/home/node/.openclaw/openclaw.json'));
    cfg.agents = cfg.agents || {};
    cfg.agents.defaults = cfg.agents.defaults || {};
    cfg.agents.defaults.skills = cfg.agents.defaults.skills || [];
    if (!cfg.agents.defaults.skills.includes('@anthropic/mcp-server-fetch')) {
      cfg.agents.defaults.skills.push('@anthropic/mcp-server-fetch');
    }
    fs.writeFileSync('/home/node/.openclaw/openclaw.json', JSON.stringify(cfg, null,
2));
    console.log('Fetch skill added');
  "

kubectl rollout restart deployment/openclaw -n openclaw
```

11. Connect Signal Messaging (Optional)

OpenClaw supports Signal as a messaging channel in addition to Discord. Signal integration requires linking OpenClaw as a secondary device to a Signal account, similar to how Signal Desktop works.

11.1 Prerequisites

- **A dedicated phone number** — a separate number (e.g., Google Voice) is recommended so the bot has its own Signal identity, separate from your personal account
- **Signal registered on a primary device** — install Signal on a phone or tablet and register with the dedicated number. You can use Google Voice numbers for SMS verification.
- **The primary device must remain active** — Signal requires at least one primary device. If you uninstall Signal from the phone, the linked account (OpenClaw) will eventually deactivate.

11.2 Register Signal with a Google Voice Number

21. Install Signal on a spare phone or tablet
22. During registration, enter your Google Voice number
23. Signal sends a verification SMS to Google Voice — check your Google Voice app or inbox for the code
24. Complete registration with the verification code
25. Once registered, keep Signal installed on this device (it must remain the primary device)

11.3 Enable Signal in OpenClaw

Enable the Signal plugin in OpenClaw's configuration. OpenClaw will generate a linking URI or QR code that you scan from the Signal app on your primary device:

```
kubectl exec -n openclaw deployment/openclaw -c openclaw -- \
  node -e "
    const fs = require('fs');
    const cfg = JSON.parse(fs.readFileSync('/home/node/.openclaw/openclaw.json'));
    cfg.plugins = cfg.plugins || {};
    cfg.plugins.entries = cfg.plugins.entries || {};
    cfg.plugins.entries.signal = { enabled: true };
    fs.writeFileSync('/home/node/.openclaw/openclaw.json', JSON.stringify(cfg, null,
2));
    console.log('Signal plugin enabled');
  "

kubectl rollout restart deployment/openclaw -n openclaw
```

11.4 Link the Device

After the pod restarts, check the logs for a Signal linking URI or QR code:

```
kubectl logs -n openclaw deployment/openclaw -c openclaw --tail=30 | grep -i signal
```

On your primary Signal device, go to Settings → Linked Devices → Link New Device, and scan the QR code or follow the linking URI from the logs. Once linked, OpenClaw will respond to Signal messages sent to the dedicated number.

> ✅ **CHECKPOINT: Signal connected Run:** kubectl logs -n openclaw deploy/openclaw -c openclaw --tail=20 | grep -i signal **Expected:** *Logs show Signal provider started and connected without errors*

12. Operational Notes

12.1 Persistent State

All OpenClaw configuration, conversation history, and installed skills are stored on the Longhorn PVC at /home/node/.openclaw. This data survives pod restarts and redeployments. The Longhorn volume is replicated across cluster nodes for redundancy.

12.2 Updating OpenClaw

OpenClaw publishes new versions frequently (roughly daily, using CalVer like v2026.2.23). To update:

```
# Pull the latest image and restart
kubectl rollout restart deployment/openclaw -n openclaw
```

Since the deployment uses image: ghcr.io/openclaw/openclaw:latest with the default imagePullPolicy, a rollout restart will pull the newest image. To pin to a specific version, replace :latest with the version tag (e.g., :2026.2.23).

12.3 Viewing Logs

```
# Recent logs
kubectl logs -n openclaw deployment/openclaw -c openclaw --tail=30

# Follow logs in real time
kubectl logs -n openclaw deployment/openclaw -c openclaw -f

# Previous container logs (after a crash)
kubectl logs -n openclaw deployment/openclaw -c openclaw --previous
```

12.4 Web UI Access

OpenClaw's gateway binds to localhost (127.0.0.1:18789) by default, which means the MetalLB LoadBalancer IP will not serve the web UI without additional configuration. For web UI access, use port-forwarding:

```
kubectl port-forward -n openclaw deployment/openclaw 18789:18789
```

Then open http://localhost:18789 in your browser and enter the gateway token found in the config:

```
kubectl exec -n openclaw deployment/openclaw -c openclaw -- \
  cat /home/node/.openclaw/openclaw.json | grep token
```

12.5 Configuration File

The OpenClaw configuration lives at /home/node/.openclaw/openclaw.json inside the container. To view or edit it:

```
# View current config
kubectl exec -n openclaw deployment/openclaw -c openclaw -- \
  cat /home/node/.openclaw/openclaw.json
```

12.6 Resource Usage

On the RK3588 ARM64 nodes, OpenClaw typically consumes 150–400m CPU at idle. On the 2026.9.x releases idle memory is about 1.1 GiB RSS (it was 400–800 Mi on early-2026 builds), and OpenClaw logs `memory pressure: level=warning` against its own 1 GiB threshold. Keep the 2Gi limit. During active conversations with tool use (web browsing, code execution), usage may spike to the 1000m/2Gi limits briefly. The 1.1 GB container image includes a Chromium browser for web automation.

13. Cleanup

To completely remove OpenClaw from your cluster:

```
# Delete all resources
kubectl delete -f openclaw-deploy.yaml

# Delete the secrets
kubectl delete secret openclaw-secrets -n openclaw

# Delete the namespace
kubectl delete namespace openclaw

# Verify everything is gone
kubectl get all -n openclaw
```

Note: Deleting the PVC will permanently destroy all OpenClaw state, conversation history, and configuration. Back up the data first if needed.

14. Quick Reference

Task	Command
Check pod status	kubectl get pods -n openclaw
View logs	kubectl logs -n openclaw deploy/openclaw -c openclaw --tail=20
Restart pod	kubectl rollout restart deploy/openclaw -n openclaw
View config	kubectl exec -n openclaw deploy/openclaw -c openclaw -- cat /home/node/.openclaw/openclaw.json
Check service IP	kubectl get svc -n openclaw
Port-forward web UI	kubectl port-forward -n openclaw deploy/openclaw 18789:18789
Check current model	kubectl logs -n openclaw deploy/openclaw -c openclaw | grep model
Check MetalLB pools	kubectl get ipaddresspool -A
Check Longhorn volume	kubectl get pvc -n openclaw

15. Troubleshooting Summary

Symptom	Cause	Fix
EACCES permission denied	Longhorn PVC owned by root, container runs as UID 1000	Ensure init container with chown -R 1000:1000 /data is present in deployment
Discord error 4014, crash loop	Message Content Intent not enabled in Discord Developer Portal	Enable all Privileged Gateway Intents, Save, then kubectl rollout restart
Pod stuck at 0/1 Running	Liveness/readiness probes failing because gateway binds to localhost	Remove probes from deployment (probes are not needed for Discord-only access)
LoadBalancer IP \ <pending>	MetalLB address-pool annotation does not match configured pool name	Check pool name: kubectl get ipaddresspool -A, fix annotation to match
Model too expensive	Default is Claude Opus (higher per-token price than Sonnet)	Follow Section 8 to switch to Sonnet; check current prices on Anthropic's pricing page
Bot not responding in Discord	Bot not invited to server, or lacks Send Messages permission	Re-invite with correct OAuth2 permissions; check channel permissions
Bot stops responding after restart	Config overwrite reset groupPolicy to allowlist	Check logs for Config overwrite line; re-run Section 8.1 to restore settings
Model reverts to Opus after restart	Config overwrite regenerated defaults	Re-apply Sonnet model per Section 8.1 (configMode no longer exists, see Section 8.3)
Web search not working	BRAVE_API_KEY not configured	Add Brave API key per Section 10; verify with bot query